

Разбор задачи «А. Автобусы»

Автор задачи — К. Батузов, автор разбора — Б. Василевский

Решение на 30 баллов

До начала ремонта дороги интервал между двумя идущими друг за другом автобусами один и тот же для всех автобусов и равен $P = \frac{L}{Mv}$.

Поскольку $v = w$, можно считать, что ремонта дороги не происходило. В этом случае ответ останется равным P .

Решение на 60 баллов

Для прохождения второй группы тестов было необходимо понять, что сразу после начала ремонта интервал между двумя соседними автобусами такой же, каким он будет на протяжении всего дальнейшего пути. Действительно, если нарисовать графики зависимости координаты на маршруте от времени, то можно увидеть следующую картину:

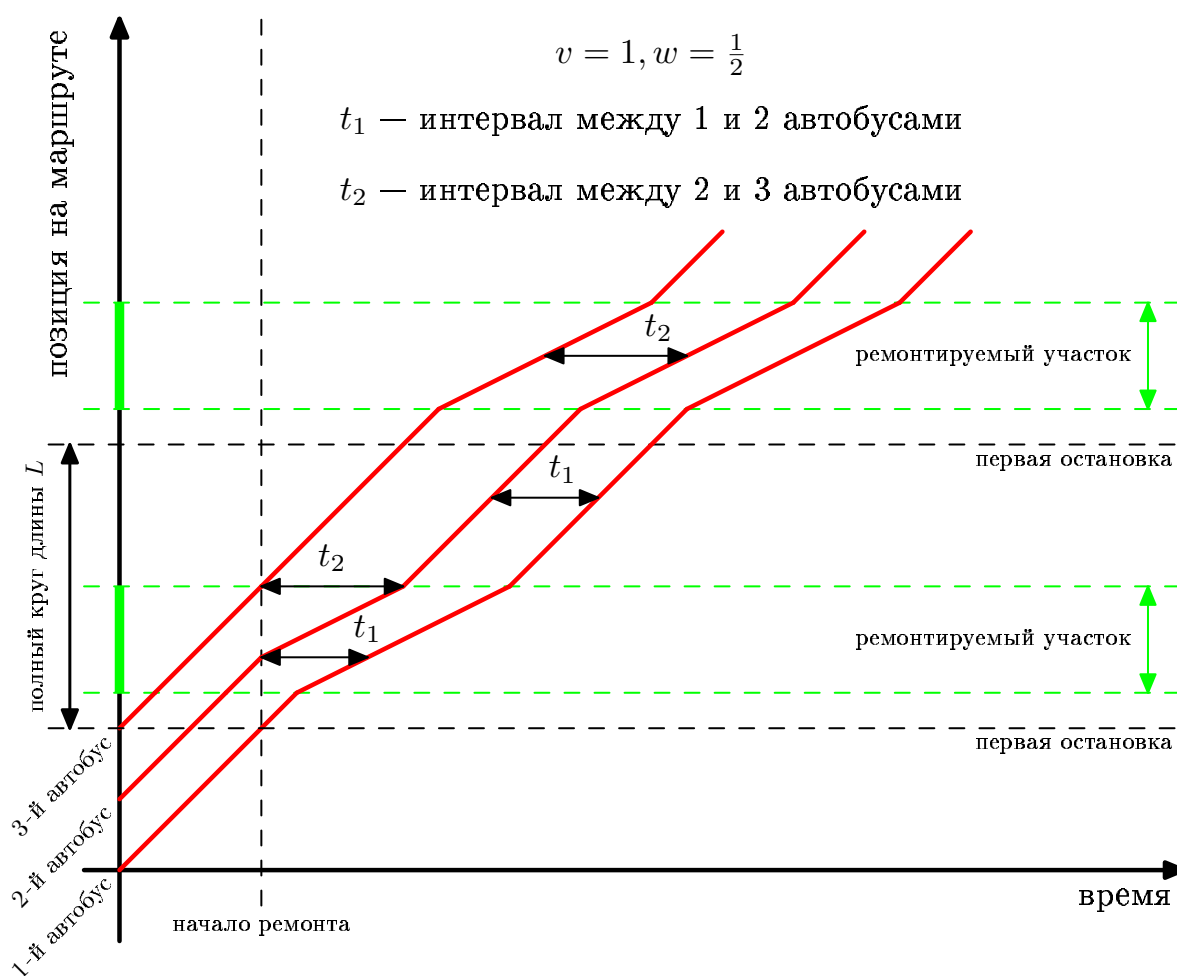


Рис. 1: Интервалы между соседними автобусами можно замерять с момента начала ремонта, ведь в дальнейшем они едут по одному и тому же закону.

Таким образом, для решения задачи нужно узнать максимальный из интервалов между соседними автобусами в тот момент времени, когда начался ремонт.

Исходя из ограничений для второй группы тестов, интервалов всего два. Первый равен в точности времени, за которое первый автобус проедет первую половину окружности (с учетом введенных скоростных ограничений). Второй определяется аналогично.

Обозначим через d_1 и d_2 длины ремонтируемых участков первой и второй половины пути соответственно, $b = \frac{1}{2}L$. Тогда первый интервал равен в точности $\frac{d_1}{w} + \frac{b-d_1}{v}$, второй — $\frac{d_2}{w} + \frac{b-d_2}{v}$. Следующий отрывок кода является содержательной частью решения, проходящего вторую группу тестов (убедитесь, что d_1 и d_2 вычисляются правильно!):

```
y = y - 1; x = x - 1 // остановки удобно нумеровать с нуля
a = L / N; // переменные a, b, d1, d2 имеют вещественный тип
b = L / M; // деление везде также вещественное
d1 = max(min(a*y, b) - a*x, 0.0);
d2 = max(a*y - max(a*x, b), 0.0);
answer = max(d1 / w + (b - d1) / v, d2 / w + (b - d2) / v);
```

Для того, чтобы набрать 60 баллов, достаточно было в случае $M > 2$ выводить ответ $P = \frac{L}{Mv}$, который работает на первой группе тестов.

Правильное решение

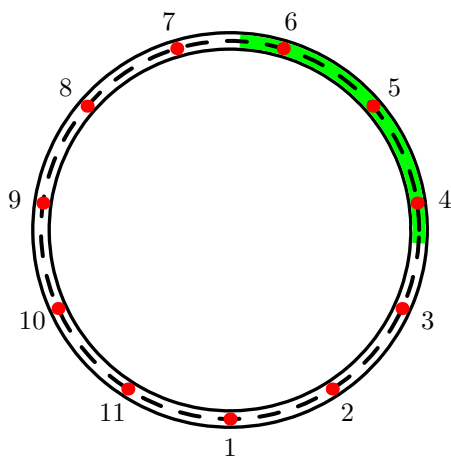


Рис. 2: Случаи 1, 2, 3, 4

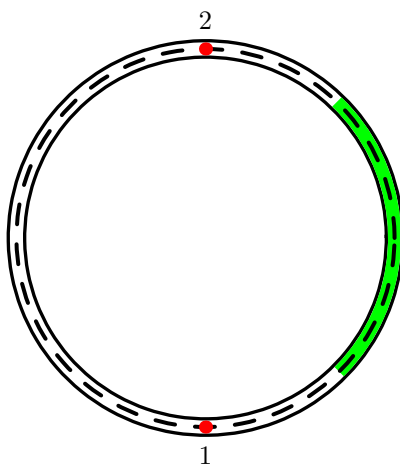


Рис. 3: Случай 5

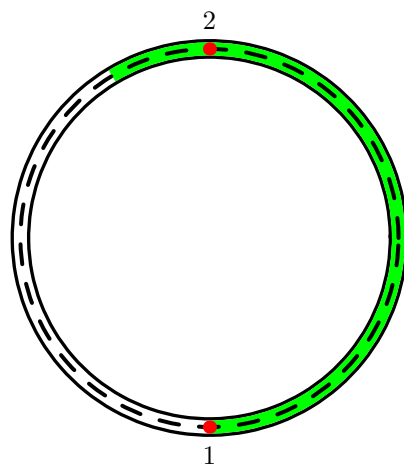


Рис. 4: Случай 4 во втором своем проявлении

Последним логическим шагом в этой задаче является следующее утверждение: всегда существует не более 5 различных интервалов. А именно, это интервалы движения между соседними автобусами, на пути между которыми в момент начала ремонта

- 1) скорость всегда равна v (рис. 2, например, $1 \rightarrow 2$; рис. 3, $2 \rightarrow 1$),
- 2) скорость всегда равна w (рис. 2, например, $4 \rightarrow 5$; рис. 4, $1 \rightarrow 2$),
- 3) скорость сначала равна v , а потом w (рис. 2, $3 \rightarrow 4$),

- 4) скорость сначала равна w , а потом v (рис. 2, $6 \rightarrow 7$; рис. 4, $2 \rightarrow 1$),
5) скорость меняется два раза и изначально равна v (рис. 3, $1 \rightarrow 2$),

Случая, когда скорость меняется два раза, но изначально равна w , быть не может, ведь для этого необходимо расположение всех автобусов строго внутри зоны ремонта. Очевидно, что первый автобус может быть либо на границе, либо вне его.

Случаи 3, 4, 5 не могут появляться более одного раза, тогда как 1 и 2, вообще говоря, составляют большинство. Поэтому для каждого из них нужно сначала проверить, что действительно ли он имеет место, и если результат положительный, то вычислить соответствующий интервал.

Несмотря на то, что возможность наличия автобусов на границе ремонтируемого участка практически никак не влияет на реализацию, за этим необходимо аккуратно проследить.

Ниже приведен код решения на си++:

```
#include <algorithm>
#include <cstdio>

using namespace std;

int l, n, m, x, y, v, w;
double answer = 0, d, a, b; // d будет обозначать длину ремонтируемого участка пути

int main()
{
    freopen("a.in", "rt", stdin);
    freopen("a.out", "wt", stdout);
    scanf("%d%d%d%d%d%d", &l, &n, &m, &x, &y, &v, &w);
    x--, y--; // теперь автобусы и остановки нумеруются с нуля

    a = l / (double)n; // расстояние между соседними автобусами
    b = l / (double)m; // расстояние между соседними остановками

    // k1 и k2 - номера первого и последнего автобусов
    // в участке ремонта в момент его начала
    long long k1 = (long long)m*x;
    k1 = k1 / n + (k1 % n != 0);
    long long k2 = (long long)m*y;
    k2 = k2 / n - (k2 % n == 0);

    if (k2 - k1 + 2 < m && m > 1) // случай 1 имеет место
        answer = max(answer, b/v);

    if (k1 < k2) // случай 2
        answer = max(answer, b/w);

    if (k1 <= k2) // в ремонтируемый участок попал хотя бы один автобус
    {
        if (k2 - k1 + 1 < m) // не все автобусы находятся строго
```

```
{ // внутри ремонтируемого участка
    // случай 3
    d = b*k1 - a*x;
    answer = max(answer, d/w + (b - d)/v);

    // случай 4
    d = a*y - b*k2;
    answer = max(answer, d/w + (b - d)/v);
}
else // случай 4, интервал между k2 и k1
{
    d = (a*y - b*k2) + (b*k1 - a*x);
    answer = max(answer, d/w + (b - d)/v);
}
}
else // случай 5
    answer = max(answer, (b - a*(y - x))/v + a*(y - x)/w);

printf("%.9lf", answer);

return 0;
}
```

Разбор задачи «В. Праздничные открытки»

Автор задачи — Е. Андреева, автор разбора — Б. Василевский

Назовем две открытки соседними, если они пересекаются хотя бы по одной клетке стола.

Сформулируем простой, но важный факт, следующий из условия. Пусть существует способ извлечь половину открыток требуемым образом. Соответствующую последовательность номеров обозначим через $a_1, a_2, \dots, a_p$, где $p = nm/2$. Тогда произвольная перестановка указанных чисел также приводит к правильному решению задачи. Другими словами, **порядок извлечения открыток не имеет значения**. Действительно, доставая очередную, мы влияем лишь на соседние с данной открытки. С другой стороны, рассматриваемая последовательность $a_1, a_2, \dots, a_p$ очевидно не может содержать соседние, ведь иначе клетка стола, по которой они пересекаются, станет в результате ненакрытой.

Решение на 30 баллов

Для прохождения первой группы тестов достаточно было написать какой-нибудь перебор.

Например, рассмотрим клетку (i, j) стола. Очевидно, что ровно одна из двух открыток, накрывающих эту ячейку, будет извлечена. В свете рассуждений в начале разбора, для каждой клетки достаточно будет перебрать оба этих варианта. Тогда придется рассмотреть 2^{NM} конфигураций, что допустимо при небольших ограничениях тестов первой группы $NM \leq 24$. Каждую из них необходимо проверить на корректность. Если противоречий не возникло, то есть каждая клетка стола накрывается ровно одной открыткой, то ответ найден.

Важно также не забывать про существование тестов с ответом NO.

Решение на 60 баллов

Рассмотрим граф, в котором вершинами будут клетки стола, а ребрами — открытки. Если открытка лежит на клетках (i_1, j_1) и (i_2, j_2) стола, то соответствующее ей ребро соединяет вершины (i_1, j_1) и (i_2, j_2) графа. Таким образом, степень каждой вершины равна двум. Если на столе имеются совпадающие открытки, то в построенном графе возникнут кратные ребра.

Назовем вершину (i, j) четной, если сумма $i + j$ четная. Все остальные вершины будем называть нечетными.

Нетрудно увидеть, что полученный граф является двудольным: первая доля состоит из четных вершин, вторая — из нечетных. Найдем максимальное паросочетание в этом графе и обозначим его размер через A . Если $A = NM/2$, то есть паросочетание совершенно, тогда выбранные ребра (открытки) искомые — удаляя их, мы оставим ровно один слой на столе. Это обусловлено тем, что теперь из каждой вершины ведет ровно одно ребро, то есть каждую клетку накрывает ровно одна открытка.

Верным является обратное утверждение: если существует способ достать некоторые открытки требуемым образом, то в нашем графе существует совершенное паросочетание, составленное из соответствующих этим открыткам ребер. Итак, если $A < NM/2$, то ответ NO.

Ограничения второй группы тестов были такими, что можно было хранить этот граф в памяти любым способом. Для поиска максимального паросочетания можно было воспользоваться алгоритмом Куна.

Правильное решение

Рассмотрим граф, описанный в решении на 60 баллов. Как уже было замечено, степень каждой вершины в нем равна двум. Поэтому наш граф представляет собой набор из нескольких непересекающихся по ребрам циклов. Для решения задачи необходимо удалить ровно $NM/2$ ребер так, чтобы степень каждой вершины стала ровно 1, или выяснить, что это невозможно.

Очевидно, что эту операцию достаточно провести независимо для каждого цикла. Рассмотрим один из них и обозначим его $\mathcal{C}$. Если он имеет нечетную длину, то искомого способа не существует. Иначе, пусть ребра этого цикла имеют номера $c_1, c_2, \dots, c_{2k}$. Ясно, что надо удалять либо все нечетные $(c_1, c_3, \dots, c_{2k-1})$, либо все четные $(c_2, c_4, \dots, c_{2k})$. Из рассуждения в начале разбора становится понятно, что необходимым и достаточным условием для возможности удаления каждой из этих последовательностей является отсутствие в ней таких ребер, что соответствующая открытка лежит снизу обеими половинками.

Итак, если у каждого из циклов $\mathcal{C}_i$ длина делится на 2, и из $\mathcal{C}_i$ можно удалить либо все четные, либо все нечетные ребра, то ответ YES. Иначе (если найдется хотя бы один плохой цикл) ответ NO.

Описанное решение работает за $O(NM)$, что допустимо при ограничениях $1 \leq N, M \leq 700$.

Разбор задачи «С. Предпраздничная суета»

Автор задачи и разбора — К. Абакумов

Решение на 30 баллов

Чтобы найти минимальное время выхода из магазина, необходимо определить, в какие кассы лучше всего становиться в очередь. Из-за небольших ограничений в 1-ой группе тестов можно использовать перебор. Рассмотрим все возможные комбинации из K касс (нам выгодно использовать максимально допустимое количество). Для каждого набора уже несложно выяснить искомое время: очередной тортик выгоднее всего пробить через ту кассу, которая на данный момент освобождается раньше. Если таких несколько, то можно выбрать любую.

Ответом будет минимальное из полученных чисел.

Решение на 60 баллов

Так как школьников в этом случае не меньше чем касс, то достаточно распределить тортики по всем кассам. Идея проста: добавлять тортик в очередь той кассы, которая на данный момент освобождается раньше.

Рассмотрим массив a из N элементов, где $a[i]$ равно времени, в которое освободится i -я касса на данный момент. Изначально $a[i] = A_i + B_i$. Чтобы решение проходило большие тесты второй группы, организуем на a кучу, позволяющую брать минимум за $O(1)$ и изменять элементы за $O(\log N)$. Добавляя текущий тортик в очередь минимальной кассы (можно выбрать любую, если таких несколько), необходимо увеличить соответствующее время. После добавления по одному P тортиков максимальное из $a[i]$ и будет ответом.

Засчет использования кучи это решение работает за $O(P \log N)$, что допустимо при ограничениях $N, P \leq 100\,000$.

Заметим, что если описанный алгоритм применять ко всем комбинациям K касс из N , то он бы успешно работал не только на второй группе тестов (так как в этом случае конфигурация только одна), но и на первой благодаря небольшим ограничениям.

Правильное решение

Основная идея решения на 100 баллов — двоичный поиск по ответу.

Предположим, что прошло ровно t единиц времени. Научимся определять, смогут ли школьники приобрести к этому моменту нужные P товаров.

Сначала для каждой кассы подсчитаем количество тортиков, которое можно купить в ней по прошествии t единиц времени (вся эта процедура займет $O(N)$ операций). Далее отсортируем подсчетом кассы по найденному количеству товаров. Эта сортировка работает за время, пропорциональное максимальному возможному из сортируемых значений. Однако, в нашем случае не имеет смысла рассматривать числа, большие $P - 1$. Действительно, если в какой-то из касс можно купить не менее P тортиков, то за время t школьники гарантированно успеют. Поэтому сортировка будет работать за $O(P)$. Осталось выбрать K максимальных значений, просуммировать их, и если полученная сумма будет больше или равна P , то школьники успеют купить все тортики к моменту времени t , иначе — нет. Суммарная сложность этой проверки равна $O(N + P)$.

Для решения задачи остается найти минимальное время, при котором школьники успеют приобрести P тортиков, методом бинарного поиска. Такое решение работает за $O((N + P) \log T)$, где T — максимально возможный ответ.

Возможные ошибки при реализации:

- Использование LongInt вместо Int64
- Деление на 0: надо учесть, что время обработки одного товара может быть равно 0

Ниже приведен текст решения на Delphi:

```
{$APPTYPE CONSOLE}
Const
  MaxN = 100000;
  MaxP = 100000;
  MaxK = 100000;

type
  Line = record //одна касса:
    w: Integer; // w - сколько надо ждать до своей очереди
    t: Integer; // t - время обработки одного товара
  end;

var
  lines: array[1..MaxN] of Line;
  sort: array[0..MaxP] of Integer; //массив для сортировки
  n, k, p, i: Integer;
  t, l, r, s: Int64;

function check(t: int64):boolean;
var
  i, S : Integer;
  X : Int64;
begin
  FillChar (sort, SizeOf(sort), 0);
  S := 0;
  //Сортировка
  for i := 1 to n do begin
    if (lines[i].t = 0) and (lines[i].w > t) then begin
      check:=true;
      Exit;
    end else
      if lines[i].t = 0 then Continue;
    x := (t-lines[i].w) div lines[i].t;
    if x<0 then Continue;
    if x>=p then begin
      check:=true;
      Exit;
    end;
  end;
```



```
    Inc(sort[x]);
end;

//Подсчет суммы
x := k;
i := 100000;
while (x > 0) and (i > 0) do begin
    if sort[i] > 0 then begin
        Dec(sort[i]);
        Inc(s,i);
        Dec(x);
    end else Dec(i);
end;

if s >= p then check:=True
else check:=false
end;

begin
    reset(input,'c.in');
    rewrite(output,'c.out');
    read(n);
    for i := 1 to n do begin
        read(lines[i].t, t, lines[i].w);
        Inc(lines[i].w, t);
    end;
    Read(k, p);
    if k > n then k := n;
    l := 0;
    r := 100000000000000;

    //Поиск времени T
    for i := 2 to n do begin
        t := Int64(lines[i].w) + int64(lines[i].t)*p;
        if t < r then r := t;
    end;

    //Двоичный поиск
    while r - l > 1 do begin
        t := (l + r) div 2;
        if check(t) then
            r := t
        else
            l := t+1;
    end;
    if check(l) then writeln(l) else writeln(r);
    close(input); close(output)
end.
```

Разбор задачи «D. Сообщающиеся сосуды»

Автор задачи Антонов Вадим

Авторы разбора Антонов Вадим, Батузов Кирилл

Перед тем, как рассматривать решения, докажем 2 леммы.

Лемма 1. Пусть в правильном решении задачи уровни жидкости в отсеках получились равными p_i . Тогда выполнены следующие утверждения:

1. $\sum_i p_i = C$
2. $\forall i \ p_i \geq p_{i+1}$
3. если $p_i > p_{i+1}$ и $p_{i+1} > 0$, то $H_i = p_i$
4. если $p_i < H_i$, то $\forall j : j > i$ будет выполнено $p_j = 0$
5. $p_i \geq 0$

Доказательство. Данные утверждения являются формализацией условия задачи. $\square$

Лемма 2. Существует единственное распределение жидкости, удовлетворяющее условиям леммы 1.

Доказательство. Пусть существуют два различных распределения жидкости p_i и q_i , удовлетворяющие условиям леммы 1.

Тогда, из 1 следует, что существуют такие индексы x и y , что, без ограничения общности $p_x > q_x$, $p_y < q_y$. Тогда найдется такой индекс i , что $x \leq i < y$ и $p_i > q_i$, $p_{i+1} \leq q_{i+1}$.

Из $p_i > q_i \geq q_{i+1} \geq p_{i+1}$ по условию 2 следует, что $p_i > p_{i+1}$ и по условию 3 получаем, что $H_i = p_i$.

Из $H_i = p_i$ и $p_i > q_i$ следует, что $q_y = 0$ (по условию 4). Но $q_y > p_y \geq 0$ (из условия 5). Противоречие. $\square$

Решение на 30 баллов

Для решения задачи на 30 баллов применим метод динамического программирования.

Предположим, что $H_0 = \infty$ и $H_N = \infty$, а в C_i будет храниться ответ для i -го столбца.

Введём два дополнительных массива: $Sum_i = \sum_{j=1}^i C_j$ и A_i^j - высота воды в столбцах между перегородками i и j после установления стационарного состояния, если в столбцах между перегородками с i по j был установлен одинаковый уровень. Из определения следует, что Sum_i не может превышать C .

Пусть посчитаны A_i^j для всех значений, меньших j . Тогда для того, чтобы посчитать A_i^j требуется вычислить $a = \min(H_i, H_j)$, $b = \max_{k \in (i,j)} H_k$ и $c = \frac{C - Sum_{i-1}}{j-i}$. Понятно, что если $b \leq a$ и $b \leq c$, то $A_i^j \in [b, a]$, а если $b > a$ или $b > c$, то такого A_i^j не существует. В случае, если A_i^j существует, то оно будет равно c , если $c \leq a$ и a — иначе (т.е. установившийся уровень воды не может быть больше, чем отверстия в перегородках i и j , и не может быть меньше, чем максимальная высота отверстия в перегородках между i и j).

Как же среди всех существующих A_i^j выбрать то, которое подходит? Исходя из лемм, нужно выбрать то A_i^j , для которого $A_i^j * (j-i) + Sum_{i-1}$ будет максимально. После определения нужного A_i^j можно посчитать Sum_j .

Исходя из этого алгоритма, понятно, что для его работы требуется проинициализировать Sum_0 нулём.

Давайте посчитаем количество операций, требуемых для этого решения. В таблице A_i^j $O(N^2)$ ячеек. Для заполнения одной ячейки требуется $O(N)$ операций. Получается $O(N^3)$.

Решение на 60 баллов

Предположим, что Петя поставил немного другой эксперимент. Пусть он поставил на каждое отверстие кран и открывал их по одному. Сначала он открыл кран в перегородке между первым и вторым отсеком, дождался пока вода достигнет состояния равновесия, затем открывал кран в перегородке между вторым и третьем отсеками, опять ждал и так далее. Легко видеть, что распределение воды, полученное в таком эксперименте удовлетворяет условиям леммы 1, а следовательно, по лемме 2, оно совпадает с распределением, которое нас просят узнать в задаче. Покажем, как его найти.

Пусть Петя уже открыл первые $L - 1$ кранов и дал воде прийти в состояние равновесия. Если изобразить уровень воды, то он будет напоминать лестницу:

$$p_1 = p_2 = \dots = p_{i_1} > p_{i_1+1} = p_{i_1+2} = \dots = p_{i_2} > \dots > p_{i_f+1} = p_{i_f+2} = \dots = p_L,$$

причем $p_{i_j} = H_{i_j} \forall j = 1..f$.

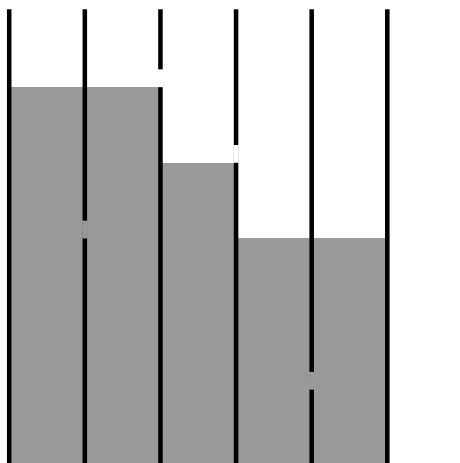


Рис. 1: Уровень воды в первых L секциях

Предположим Петя открыл кран, между отсеками L и $L + 1$. При этом вода начнет перетекать из последней «ступеньки» в секцию $L + 1$, а уровень воды в других «ступеньках» (то есть в секциях с первой по i_f) меняться не будет. При перетекании из последней «ступеньки» в новый отсек уровень воды секциях последней «ступеньки» будет равномерно уменьшаться до тех пор, пока не произойдет одно из следующих событий:

1. уровень воды в секции $L + 1$ и в секциях последней «ступеньки» выровняется
2. уровень воды в «ступеньке» достигнет одного из кранов, расположенных внутри нее (мы будем считать, что кран между последней «ступенькой» и следующей за ней секцией также расположен внутри последней «ступеньки»)

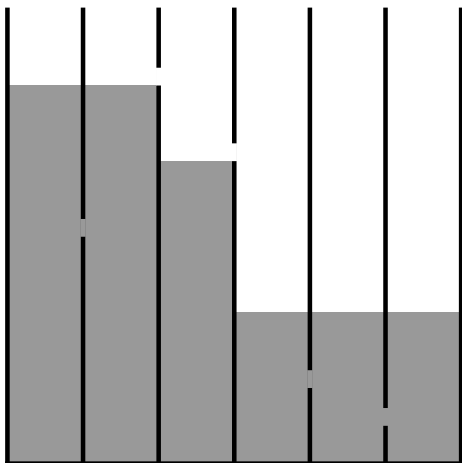


Рис. 2: Случай 1

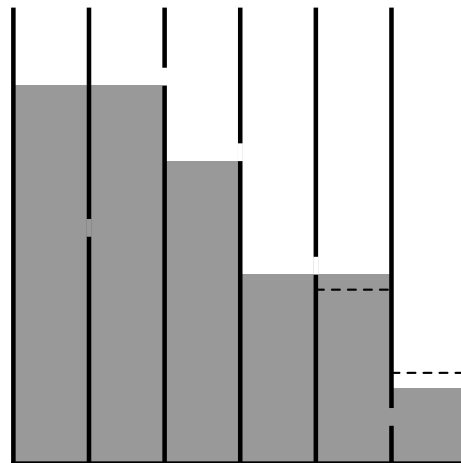


Рис. 3: Случай 2

Рис. 4: Уровень воды в случае 1 (рис. 2) и 2 (рис. 3). Во втором случае пунктиром показано изменение уровня воды в дальнейшем.

В первом случае вода достигла равновесия и Петя может окрывать следующий кран. Во втором случае из последней «ступеньки» образуются две новых. Из последней из них вода продолжит перетекать в секцию $L + 1$ по тому же закону.

Сформулируем алгоритм. Начальное состояние — одна «ступенька» из первой секции с уровнем воды C , открыт один кран.

Шаг алгоритма:

1. Вычислить, сколько воды должно перетечь из «ступеньки» в следующую секцию до наступления события 1.
2. Вычислить, сколько воды должно перетечь из «ступеньки» в следующую секцию до наступления события 2.
3. Если событие 1 наступит раньше, чем событие 2, то расширить «ступеньку» на одну секцию, пересчитать уровень воды, открыть следующий кран.
4. Если событие 2 наступит раньше, чем событие 1, то разбить «ступеньку» на две, пересчитать уровень воды в этих ступеньках и в следующей секции.

Шаг необходимо повторять до тех пор, пока не будет получен окончательный ответ.

Очевидно, что в ходе алгоритма границы последней ступеньки движутся только вправо, причем на каждом шаге одна из них изменяется. Значит алгоритм выполнит $O(N)$ шагов.

Пункт 1 шага алгоритма легко реализовать за $O(1)$ — необходимо решить линейной уравнение с одной неизвестной. В пункте 2 необходимо найти кран, расположенный наиболее высоко внутри последней «ступеньки» (именно по нему она будет разделена). Простая реализация требует $O(N)$ операций. Пункты 3 и 4 также легко реализуются за $O(N)$ операций, поскольку необходимо записать новый уровень воды во все секции. Таким образом, при реализации данного алгоритма будет работать за $O(N^2)$ операций.

Правильное решение

Покажем, как реализовать предыдущее решение за $O(N)$.

Начнем с пунктов 3–4. Заметим, что в ходе шага алгоритма уровень воды изменяется только в секциях, входящих в состав последней «ступеньки», и следующей за ней секции. Причем, уровень воды во всех секциях, входящих в состав «ступеньки», одинаков. Значит, нам достаточно хранить этот уровень, а в ответ мы его запишем в пункте 4 в те секции, которые перейдут из последней «ступеньки» во вновьобразованную предпоследнюю. Уровень воды во всех «ступеньках», кроме последней в ходе алгоритма изменяться не может, поэтому на запись ответа суммарно будет затрачено $O(N)$ времени. Все остальные действия в пунктах 3–4 выполняются за $O(1)$.

Остался пункт 2. Предположим, что у нас есть массив h :

- h_1 — кран, расположенный внутри последней «ступеньки» и имеющий наибольшую высоту. Среди всех таких — самый правый.
- h_i , где $i > 1$ — кран, расположенный внутри последней «ступеньки» правее крана h_{i-1} и имеющий наибольшую высоту. Среди всех таких — самый правый.

Первый элемент массива h позволит реализовать пункт 2 за $O(1)$. Однако возникает другая проблема — в пунктах 3 и 4 необходимо пересчитывать этот массив. Покажем, как это сделать эффективно.

С пунктом 4 все просто — необходимо отбросить первый элемент массива. Мы, конечно, не будем двигать содержимое массива, а просто сдвинем его начало. Это требует $O(1)$ времени.

В пункте 3 воспользуемся идеей, аналогичной идее, используемой в алгоритме Грэхема построения выпуклой оболочки. Пусть при добавлении новой секции к «ступеньке» добавился кран W . Пока последний кран в массиве h расположен ниже крана W будем удалять последний элемент массива h . Затем добавим W в конец массива h . Удаление одного элемента из конца массива требует $O(1)$ времени. Поскольку за каждый шаг алгоритма в массив h добавляется не более одного элемента, а всего шагов $O(N)$, то суммарно на все удаления будет затрачено $O(N)$ времени.

Таким образом весь алгоритм будет работать за $O(N)$.

Разбор задачи «Е. Верное равенство»

Автор задачи — В. Матюхин, автор разбора — Д. Елишин

Запишем числа A , B и C как при сложении столбиком, друг под другом.

Решение на 30 баллов

По условию, при сложении их в искомой системе счисления не происходит переноса разряда. Кроме того, записи A , B и C состоят из цифр от 0 до 9. Из этого можно сделать вывод, что в каждом столбце сумма цифр, относящихся к A и B , равна соответствующей цифре в записи C . Поэтому искомая система счисления на единицу больше максимальной из цифр в записи данных чисел.

Все тесты этой группы имели решение, поскольку в ограничениях ясно говорилось о его существовании.

Решение на 60 баллов

Будем перебирать основания системы счисления K и проверять выполнение равенства $A + B = C$ в каждой из них. Основная идея заключается в том, что не имеет смысла рассматривать $K \geq 20$. Действительно, в случае $K = 19$ при сложении не может происходить переноса разряда, ведь самый плохой случай — это $9 + 9$. Поэтому проверка равенства заключается в сравнении независимо в каждом столбце суммы двух верхних цифр и нижней. Та же ситуация будет и при $K > 19$. Таким образом, если не существует ответа меньше 20, то его не существует вообще.

Остается правильно реализовать этот перебор. Важно не забывать о том, что при переводе чисел в системы с большими основаниями результат может не вмещаться в `longint`, и нужно использовать `int64`.

Правильное решение

Здесь не добавляется принципиально новых идей. Необходимо лишь научиться работать с длинными числами. Пусть задано $2 \leq K \leq 19$. Как в алгоритме сложения столбиком, будем проверять совпадение соответствующих чисел, не забывая хранить в дополнительной переменной наличие/отсутствие переноса разряда.

Разбор задачи «F. Симпатичные прямоугольники»

Автор задачи и разбора — М. Тихомиров

Решение на 30 баллов

Будем проверять каждый подпрямоугольник на симпатичность и считать количество подходящих. Для этого переберём все возможные части четырьмя вложенными циклами по координатам границ подпрямоугольника. Непосредственным сравнением символов, симметричных относительно центра текущей части, проверим её на симпатичность.

Посчитаем общее количество подпрямоугольников в картине. Каждый из них задается четырьмя числами — координатами верхней и нижней (от 1 до N), а также левой и правой границы (от 1 до M). Поскольку координата верхней границы не должна превышать координаты нижней, возможных комбинаций этих двух чисел $N(N-1)/2 = O(N^2)$. Аналогично для левой и правой границы — $O(M^2)$. Таким образом, всего возможных кусков $O((NM)^2)$.

Количество операций, необходимых для проверки одного куска, равно половине количества символов в нём, что равно $O(NM)$. Перемножая, получаем общую сложность алгоритма — $O((NM)^3)$.

Решение на 60 баллов

В предыдущем решении мы проверяли части на симпатичность независимо друг от друга. Оптимизируем алгоритм за счёт использования уже полученных результатов.

Пусть нам необходимо проверить некоторую часть A , ширина которой больше двух. Рассмотрим прямоугольник B , получаемый из A отрезанием левого и правого крайних столбцов. Несложно заметить, что центры A и B совпадают. Поэтому симпатичность B является необходимым условием симпатичности A . Чтобы завершить проверку, остаётся проверить симметричные буквы из крайних столбцов прямоугольника A .

Итак, два внешних цикла будут по верхней и нижней границам проверяемых кусков. Внутри них мы можем, постепенно увеличивая ширину, проверять каждый новый кусок за $O(N)$. Итоговая сложность — $O(N^3M^2)$.

Код решения (на C++):

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    freopen("f.in", "rt", stdin);
    freopen("f.out", "wt", stdout);
    int n, m;
    cin >> n >> m;
    string a[100];
    for (int i = 0; i < n; ++i)
        cin >> a[i];
    int ans = 0;
```

```
for (int x1 = 0; x1 < n; ++x1)
    for (int x2 = x1; x2 < n; ++x2)
    {
        for (int y = 0; y < m; ++y)
        {
            int f = 1;
            for (int d = 0; (y - d >= 0 && y + d < m); ++d)
            {
                for (int x = x1; x <= x2; ++x)
                    f &= (a[x][y - d] == a[x2 - (x - x1)][y + d]);
                if (!f) break;
                ans++;
            }
        }
        for (int y = 1; y < m; ++y)
        {
            int f = 1;
            for (int d = 1; (y - d >= 0 && y + d - 1 < m); ++d)
            {
                for (int x = x1; x <= x2; ++x)
                    f &= (a[x][y - d] == a[x2 - (x - x1)][y + d - 1]);
                if (!f) break;
                ans++;
            }
        }
    }
cout << ans << endl;
return 0;
}
```

Правильное решение

В этом пункте мы научимся проверять часть A на симпатичность за $O(1)$. Для этого рассмотрим два ее подпрямоугольника. Первый, B , как и до этого получается из A отрезанием левого и правого крайних столбцов. Второй, C , получается отрезанием верхней и нижней крайних строк. Опять же, для симпатичности A необходимо и достаточно выполнение двух условий:

- B и C симпатичные,
- противоположные угловые клетки A равны.

Эта идея позволяет проверять каждый кусок за $O(1)$, что дает итоговую сложность $O((NM)^2)$. Основная трудность реализации заключается в том, что в каждый момент времени надо иметь информацию о симпатичности частей B и C . При этом хранить каждый прямоугольник в памяти явно невозможно: 100^4 байт не укладывается в ограничения задачи). Проблема решается несколькими способами:

- Независимые вычисления для каждого центра (в этом случае расход памяти квадратичный).

- Поскольку общее количество подпрямоугольников не может быть больше $(100 * 99 / 2)^2$, то эффективной их нумерацией можно добиться уменьшения размера массива в 4 раза.
- Использование битовых массивов.

Релизация третьего подхода:

```
#include <iostream>
using namespace std;

char b[13][100][100][100] = {{{{0}}}};

void place1(int x1, int x2, int y1, int y2)
{
    b[x1>>3][x2][y1][y2] += 1<<(x1&7);
    return;
}

bool get(int x1, int x2, int y1, int y2)
{
    if (x1 > x2 || y1 > y2) return 1;
    return (b[x1>>3][x2][y1][y2] & (1 << (x1 & 7)));
}

int main()
{
    int n, m;
    freopen("f.in", "rt", stdin);
    freopen("f.out", "wt", stdout);
    cin >> n >> m;
    string a[100];
    for (int i = 0; i < n; ++i)
        cin >> a[i];
    int ans = 0;
    for (int dx = 1; dx <= n; ++dx)
        for (int dy = 1; dy <= m; ++dy)
            for (int x = 0; x + dx <= n; ++x)
                for (int y = 0; y + dy <= m; ++y)
                    if (get(x+1, x+dx-1, y, y+dy-1) && get(x, x+dx-1, y+1, y+dy-2) &&
                        a[x][y] == a[x+dx-1][y+dy-1] && a[x][y+dy-1] == a[x+dx-1][y])
                    {
                        place1(x, x+dx-1, y, y+dy-1);
                        ans++;
                    }
    cout << ans << endl;
    return 0;
}
```

Решение на $+\infty$

Для дальнейшей оптимизации решения используется полиномиальное хэширование строк. За $O(NM)$ насчитаем хэш для всех префиксов всех строк картины, а также для префиксов инвертированных строк. Теперь мы за $O(1)$ можем получить хэш подстроки любой строки и любой инвертированной строки. Про полиномиальное хеширование можно прочитать, например, здесь:

http://e-maxx.ru/algo/string_hashes

Внешний цикл будет по координатам центра проверяемых частей. Рассмотрим отдельно случаи чётной и нечётной высоты кусков (без ограничения общности — нечётной). Найдём максимальную строку с данным центром, являющуюся симпатичной частью (то есть палиндромом). Увеличивая высоту на 2 (прибавляя по строке сверху и снизу), каждый раз будем определять максимальную длину части с такой высотой, являющейся симпатичной. Это число полностью зависит от добавленных строк. Сравнивая хэши прямой верхней и инвертированной нижней строк, будем уменьшать длину на 2 до тех пор, пока они не станут равными. Количество симпатичных частей с данным центром и данной высотой будет равно $(k + 1)/2$, где k — найденная длина максимальной части, а $/$ — целочисленное деление. Прибавим эту величину к ответу.

Для чётной высоты следует начинать с частей высоты 2.

Поскольку при увеличении высоты максимальная длина не увеличивается, для обработки каждого центра потребуется $O(N + M)$ операций. Общая сложность алгоритма равна $O(NM(N + M))$.

Код решения (автор — Сергей Копелиович):

```
#include <cstdio>
#include <cstring>
#include <iostream>

using namespace std;

typedef long long ll;

#define forn(i, n) for (int i = 0; i < (int)(n); i++)
#define P 239017
#define maxn 100

int h, w, eq[2][maxn][maxn];
char s[maxn][maxn + 1];
ll h1[maxn][maxn + 1];
ll h2[maxn][maxn + 1];
ll p[maxn + 1];

inline ll H1( int i, int l, int r )
{
    return h1[i][r + 1] - h1[i][l] * p[r - l + 1];
}
```

```
inline ll H2( int i, int l, int r )
{
    return h2[i][l] - h2[i][r + 1] * p[r - l + 1];
}

int main()
{
    p[0] = 1;
    forn(i, maxn)
        p[i + 1] = p[i] * P;

    freopen("f.in", "r", stdin);
    freopen("f.out", "w", stdout);

    scanf("%d%d", &h, &w);
    forn(i, h)
        scanf("%s", s[i]);

    memset(h1, 0, sizeof(h1));
    forn(i, h)
        forn(j, w)
            h1[i][j + 1] = h1[i][j] * P + s[i][j];
    memset(h2, 0, sizeof(h2));
    forn(i, h)
        for (int j = w - 1; j >= 0; j--)
            h2[i][j] = h2[i][j + 1] * P + s[i][j];

    ll sum = 0;
    forn(t1, 2)
        forn(t2, 2)
            forn(i, h)
                forn(j, w)
                    {
                        int k = w;
                        while (!(0 <= j - k && j + k + t2 < w))
                            k--;
                        for (int l = 0; 0 <= i - l && i + l + t1 < h; l++)
                            {
                                while (k >= 0 &&
                                    H1(i - l, j - k, j + k + t2) != H2(i + l + t1, j - k, j + k + t2))
                                    k--;
                                sum += k + 1;
                            }
                    }
            cout << sum << endl;
    return 0;
}
```

Разбор задачи «G. Шестеренки»

Автор задачи и разбора — Иван Богатый

Решение на 30 баллов

Для получения 30 баллов достаточно было запрограммировать перебор всех троек, которые можно получить из первой, и проверить, есть ли среди них вторая. Для этого нужно моделировать один поворот шестеренок до тех пор, пока мы не встретим или ту тройку, с которой начинали (и тогда ответ NO), или ту, которую хотим получить (и тогда ответ YES).

Решение на 60 баллов

Пусть x — количество поворотов, которое необходимо сделать, чтобы перейти от первой конфигурации ко второй, если это возможно. Из условия задачи, мы имеем следующие три соотношения на x :

$$\begin{cases} x \equiv a_1 - b_1 \pmod{n}, \\ x \equiv b_2 - a_2 \pmod{m}, \\ x \equiv a_3 - b_3 \pmod{k}. \end{cases} \quad (1)$$

Чтобы ответить на вопрос задачи, надо всего лишь узнать, имеет ли данная система решение. Для этого необходимо и достаточно проверить ее по модулю каждого простого делителя n, m, k . Более точно, если $n = n' \cdot p^{e_1}, m = m' \cdot p^{e_2}, k = k' \cdot p^{e_3}$, и e_1, e_2, e_3 максимальны, то должно выполняться

$$\begin{cases} x \equiv a_1 - b_1 \pmod{p^{e_1}}, \\ x \equiv b_2 - a_2 \pmod{p^{e_2}}, \\ x \equiv a_3 - b_3 \pmod{p^{e_3}} \end{cases} \quad (2)$$

для всех простых p .

Проверить совместность (существование решения) такой системы нетрудно: x определяется из сравнения, в котором e_i максимально. Полученное значение надо подставить в два других сравнения, и убедиться, что не возникло противоречий.

Если, например, $e_1 \geq e_2$ и $e_1 \geq e_3$, то совместность системы (2) равносильна выполнению равенств $a_1 - b_1 \equiv b_2 - a_2 \pmod{p^{e_2}}$ и $a_1 - b_1 \equiv a_3 - b_3 \pmod{p^{e_3}}$.

По китайской теореме об остатках, начальная система (1) имеет решение тогда и только тогда, когда для всех простых p системы (2) совместны. Естественно, имеет смысл проверять лишь те p , которые являются делителями хотя бы одного из n, m, k . Таким образом, мы получаем алгоритм, работающий за время разложения n, m, k на простые, т.е. укладывающийся в ограничения $n, m, k \leq 10^9$.

Примечание. Рассмотрим произвольную систему из двух сравнений

$$\begin{cases} x \equiv a \pmod{n}, \\ x \equiv b \pmod{m}. \end{cases} \quad (3)$$

Обозначим $g = \gcd(n, m)$ (наибольший общий делитель). Тогда система (3) равносильна

следующей:

$$\begin{cases} x \equiv a \pmod{g}, \\ x \equiv a \pmod{n/g}, \\ x \equiv b \pmod{g}, \\ x \equiv b \pmod{m/g}, \end{cases} \quad (4)$$

а она совместна тогда и только тогда, когда $a \equiv b \pmod{g}$.

Используя описанный механизм, исходную систему (1) можно свести к системе из, вообще говоря, девяти сравнений с попарно взаимно простыми модулями (существование решения которой гарантирует китайская теорема об остатках), либо обнаружить ее несовместность. Так можно было решить задачу на 100 баллов, однако здесь мы рассмотрим другой подход, дающий более простой способ решения исходной задачи.

Правильное решение

Докажем, что две тройки (a_1, a_2, a_3) и (b_1, b_2, b_3) принадлежат одной последовательности поворотов тогда и только тогда, когда удовлетворяют системе сравнений

$$\begin{cases} a_2 + a_1 \equiv b_2 + b_1 \pmod{\gcd(m, n)}, \\ a_2 + a_3 \equiv b_2 + b_3 \pmod{\gcd(m, k)}, \\ a_1 - a_3 \equiv b_1 - b_3 \pmod{\gcd(n, k)}. \end{cases} \quad (5)$$

Или, эквивалентно:

$$\begin{cases} a_2 + a_1 \equiv b_2 + b_1 \pmod{\gcd(m, n)}, \\ a_2 + a_3 \equiv b_2 + b_3 \pmod{\gcd(m, k)}, \\ a_1 - a_3 \equiv b_1 - b_3 \pmod{\gcd(n, k)/\gcd(n, m, k)}. \end{cases}$$

При этом мы неявно получим еще один способ решения с помощью расширенного алгоритма Евклида.

Необходимость. При одном повороте на первой шестеренке вместо числа a_1 возникает число $a_1 - 1$ или $a_1 - 1 + n$, на второй шестеренке вместо числа a_2 возникает число $a_2 + 1$ или $a_2 + 1 - m$, на третьей шестеренке вместо числа a_3 возникает число $a_3 - 1$ или $a_3 - 1 + k$. Видно, что при одном повороте, а значит и при любом их количестве, величины

$$a_2 + a_1 \pmod{\gcd(m, n)}, \quad a_2 + a_3 \pmod{\gcd(m, k)}, \quad a_1 - a_3 \pmod{\gcd(n, k)}$$

не меняются.

Достаточность.

Назовем *канонической* любую тройку вида $(1, \tilde{a}_2, \tilde{a}_3)$, где $1 \leq \tilde{a}_2 \leq \gcd(n, m)$ и $1 \leq \tilde{a}_3 \leq \gcd(\text{lcm}(n, m), k)$ (lcm — наименьшее общее кратное).

Докажем, что сделав правильно подобранное число поворотов, конфигурацию (a_1, a_2, a_3) можно перевести в некоторую каноническую $(1, \tilde{a}_2, \tilde{a}_3)$. Сделав сперва $a_1 - 1$ поворотов, мы придем к эквивалентной тройке $(1, \bar{a}_2, \bar{a}_3)$. По алгоритму Евклида существуют такие целые числа x и y , что $\gcd(n, m) = ym - xn$. Тогда после совершения xn поворотов число на первой шестеренке не изменится, а число на второй шестеренке уменьшится ровно на $\gcd(n, m)$. Опять же, по алгоритму Евклида существуют такие целые числа z и t , что $\gcd(\text{lcm}(n, m), k) = z \text{lcm}(n, m) - tk$. Тогда после совершения $z \text{lcm}(n, m)n$ поворотов числа

на первой и второй шестеренках не изменятся, а число на третьей уменьшится ровно на $\gcd(\text{lcm}(n, m), k)$. Таким образом, мы можем «подкрутить» конфигурацию $(1, \bar{a}_2, \bar{a}_3)$ так, чтобы получить из нее каноническую.

Полученная тройка $(1, \tilde{a}_2, \tilde{a}_3)$ эквивалентна первоначальной тройке, поэтому связана с ней сравнениями (5). Докажем, что система (5) определяет ее однозначно. Действительно, пусть еще одна каноническая тройка $(1, \hat{a}_2, \hat{a}_3)$ удовлетворяет системе сравнений (5). Тогда

$$\begin{cases} \tilde{a}_2 \equiv \hat{a}_2 \pmod{\gcd(m, n)}, \\ \tilde{a}_2 + \tilde{a}_3 \equiv \hat{a}_2 + \hat{a}_3 \pmod{\gcd(m, k)}, \\ \tilde{a}_3 \equiv \hat{a}_3 \pmod{\gcd(n, k)}. \end{cases}$$

Из первого сравнения и ограничений на числа $\tilde{a}_2$ и $\hat{a}_2$ вытекает равенство $\tilde{a}_2 = \hat{a}_2$. Поэтому два последних равенства из системы сравнений принимают вид

$$\begin{cases} \tilde{a}_3 \equiv \hat{a}_3 \pmod{\gcd(m, k)}, \\ \tilde{a}_3 \equiv \hat{a}_3 \pmod{\gcd(n, k)}, \end{cases}$$

что эквивалентно одному сравнению

$$\tilde{a}_3 \equiv \hat{a}_3 \pmod{\text{lcm}(\gcd(m, k), \gcd(n, k))}.$$

А так как $\text{lcm}(\gcd(m, k), \gcd(n, k)) = \gcd(\text{lcm}(n, m), k)$, то $\tilde{a}_3 = \hat{a}_3$.

Итак, мы показали, что для любой тройки (a_1, a_2, a_3) существует единственная эквивалентная ей каноническая тройка. Поэтому конфигурации (a_1, a_2, a_3) и (b_1, b_2, b_3) принадлежат одной последовательности тогда и только тогда, когда эквивалентные им канонические тройки совпадают, что равносильно выполнению для них системы (5). Остается воспользоваться предположением о том, что система (5) выполнена для исходных конфигураций.

Разбор задачи «Н. Великий Флатландский Забор»

Авторы задачи — жюри олимпиады, автор разбора — А. Ляхно

Решение на 30 баллов

Отметим, что решение можно искать в классе K -угольников. Если существует Забор, содержащий менее K башен и ограничивающий площадь S , то существует и Забор, содержащий ровно K башен и ограничивающий площадь не менее S .

Забор однозначно задаётся номерами включаемых в него башен. При условии $N \leq 20$ ограничение по времени позволяет явно перебрать все сочетания из N по K , задающие возможные варианты построения Забора. Поскольку в случае выпуклой границы все диагонали целиком лежат внутри Флатландии, то для решения задачи достаточно из всех задаваемых сочетаниями вариантов выбрать тот, которому соответствует многоугольник наибольшей площади. Асимптотика такого решения составляет $O(C_N^K \cdot K)$.

Решение на 60 баллов

В случае невыпуклой границы некоторые диагонали могут выходить за пределы Флатландии. Из множества всех сочетаний из N по K , задающих варианты построения Забора, необходимо рассматривать только те, которые не содержат таких диагоналей.

Диагональ (P_i, P_j) , $1 \leq i < j \leq N$, целиком лежит внутри многоугольника $(P_1, P_2, \dots, P_N)$, если выполнены следующие три условия:

- Сумма площадей частей $(P_i, P_{i+1}, \dots, P_j)$ и $(P_j, P_{j+1}, \dots, P_N, P_1, \dots, P_i)$ равна площади всего многоугольника $(P_1, P_2, \dots, P_N)$.
- Диагональ не пересекает ни одну из сторон многоугольника, за исключением, быть может, пересечений по вершинам. Ситуация, когда сторона целиком лежит на диагонали, также не считается за пересечение.
- Рассмотрим множество вершин, лежащих на диагонали (P_i, P_j) за исключением самих P_i и P_j . Среди смежных к ним не должно существовать вершин, лежащих по разные стороны от прямой (P_i, P_j) .

Проверка этих условий для одной диагонали производится за $O(N)$. Соответственно проверка всех диагоналей может быть реализована за $O(N^3)$. Общая асимптотика решения составляет $O(N^3 + C_N^K \cdot K)$.

Правильное решение

Рассмотрим граф G , в котором вершинам соответствуют башни, а рёбрам — диагонали, их соединяющие. Граф является ориентированным — диагонали (P_i, P_j) соответствуют сразу два ребра: $(i \rightarrow j)$ и $(j \rightarrow i)$. Вес ребра $(i \rightarrow j)$ определим как площадь отсекаемой диагональю (P_i, P_j) части Флатландии. Отсекаемой является та часть, которая остаётся по правую сторону от диагонали при движении от башни i к башне j .

В вышеописанных терминах задача построения оптимального Забора сводится к нахождению в графе G цикла минимального веса, содержащего ровно K рёбер. Обозначим через $A_{N \times N}^k$ матрицу длин кратчайших путей, содержащих ровно k рёбер. Элемент a_{ij}^k содержит длину кратчайшего пути из вершины i в вершину j . В частности, A^1 совпадает с матрицей весов рёбер графа G . Матрица A^K может быть вычислена при помощи соотношения $A^k = A^{k-1} \times A^1$, где операция умножения $C = A \times B$ задаётся следующей формулой:

$$c_{ij} = \min_k (a_{ik} + b_{kj}).$$

Площадь, ограничиваемая оптимальным Забором, равна $\min(a_{ii}^K | i = 1 \dots N)$.

Время перемножения двух матриц размером $N \times N$ составляет порядка $O(N^3)$. Итоговая асимптотика описанного решения — $O(N^3 \cdot K)$. Если же для вычисления A^k вместо соотношения $A^k = A^{k-1} \times A^1$ использовать алгоритм быстрого возведения в степень, то задача может быть решена за $O(N^3 \log K)$.

Ниже приведён текст решения с использованием алгоритма быстрого возведения в степень:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>

const char infile[] = "h.in";
const char outfile[] = "h.out";

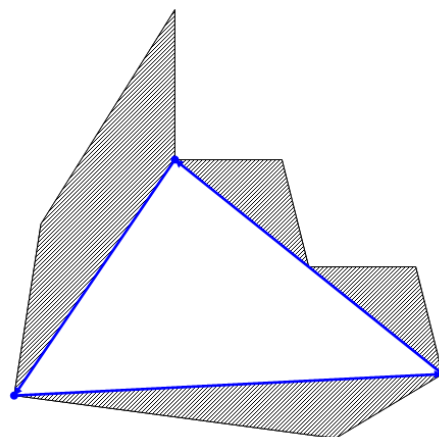
const int NMAX = 305;
const int MMAX = 20;
const int INF = INT_MAX / 2;

struct Point
{
    int x, y;
};

Point border[2*NMAX];
int square;
int n, k;

int a[MMAX][NMAX][NMAX];
int b[MMAX][NMAX][NMAX];
int prev[MMAX];
int m = 1;

int Vc(const Point &p1, const Point &p2)
```




```
{
    return p1.x * p2.y - p1.y * p2.x;
}

int Vc(const Point &p1, const Point &p2, const Point &p3, const Point &p4)
{
    return (p2.x - p1.x) * (p4.y - p3.y) - (p2.y - p1.y) * (p4.x - p3.x);
}

int Sc(const Point &p1, const Point &p2, const Point &p3, const Point &p4)
{
    return (p2.x - p1.x) * (p4.x - p3.x) + (p2.y - p1.y) * (p4.y - p3.y);
}

int Sign(int x)
{
    if (x > 0)
        return 1;
    else if (x == 0)
        return 0;
    else
        return -1;
}

void BuildFenceSegment(int i, int j)
{
    int s1 = Vc(border[j], border[i]);
    for (int k = i; k < j; k++)
        s1 += Vc(border[k], border[k + 1]);

    int s2 = Vc(border[i], border[j]);
    for (int k = j; k < i + n; k++)
        s2 += Vc(border[k], border[k + 1]);

    bool ok = abs(s1) + abs(s2) == square;

    bool sn[3];
    memset(sn, 0, sizeof(sn));

    if (ok)
        for (int k = 0; k < n; k++)
            if (Vc(border[i], border[j], border[i], border[k]) == 0
                && Sc(border[k], border[i], border[k], border[j]) < 0)
            {
                int k1 = k > 0 ? k - 1 : n - 1;
                int k2 = k + 1;

                sn[Sign(Vc(border[i], border[j], border[i], border[k1])) + 1] = true;
            }
}
```

```
        sn[Sign(Vc(border[i], border[j], border[i], border[k2])) + 1] = true;
        if (sn[0] && sn[2])
        {
            ok = false;
            break;
        }
    }

    if (ok)
        for (int k = 0; k < n; k++)
            if (Sign(Vc(border[k], border[k + 1], border[k], border[i])) *
                Sign(Vc(border[k], border[k + 1], border[k], border[j])) < 0
                && Sign(Vc(border[i], border[j], border[i], border[k])) *
                Sign(Vc(border[i], border[j], border[i], border[k + 1])) < 0)
            {
                ok = false;
                break;
            }

    if (ok)
    {
        a[0][i][j] = s1;
        a[0][j][i] = s2;
    }
    else
    {
        a[0][i][j] = INF;
        a[0][j][i] = INF;
    }
}

void Mul(int w1[][NMAX], int w2[][NMAX], int r[][NMAX], int p[][NMAX])
{
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
        {
            r[i][j] = INF;
            for (int k = 0; k < n; k++)
                if (r[i][j] >= w1[i][k] + w2[k][j])
                {
                    r[i][j] = w1[i][k] + w2[k][j];
                    p[i][j] = k;
                }
        }
}
```

```
void RestoreFence(int m, int i, int j)
{
    if (m > 0)
    {
        RestoreFence(m - 1, i, b[m][i][j]);
        printf("%d ", b[m][i][j] + 1);
        RestoreFence(prev[m], b[m][i][j], j);
    }
}

int main()
{
    freopen(inpfile, "r", stdin);
    freopen(outfile, "w", stdout);

    scanf("%d%d", &n, &k);
    for (int i = 0; i < n; i++)
    {
        scanf("%d%d", &border[i].x, &border[i].y);
        border[i + n] = border[i];
    }

    square = 0;
    for (int i = 0; i < n; i++)
        square += Vc(border[i], border[i + 1]);

    for (int i = 0; i < n; i++)
        a[0][i][i] = INF;

    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            BuildFenceSegment(i, j);

    int r = 1;
    while (k >> r)
        r++;

    for (int h = r - 2; h >= 0; h--)
    {
        Mul(a[m - 1], a[m - 1], a[m], b[m]);
        prev[m] = m - 1;
        m++;

        if (k & (1 << h))
        {
            Mul(a[m - 1], a[0], a[m], b[m]);
            prev[m] = 0;
            m++;
        }
    }
}
```

```
    }  
}  
  
int res = INF;  
int v = 0;  
  
for (int i = 0; i < n; i++)  
    if (a[m - 1][i][i] < res)  
    {  
        res = a[m - 1][i][i];  
        v = i;  
    }  
  
printf("%.5lf\n", (square - res) / 2.0);  
printf("%d\n", k);  
  
printf("%d ", v + 1);  
RestoreFence(m - 1, v, v);  
  
return 0;  
}
```